Operating Systems
UE24CS242B

OS Orange Problem - 2

Name: Ravitejas
SRN: PES1UG24CS603
Class: 4J
GitHub: https://github.com/R4vitejas/PES1UG24CS603-pes-vcs

Phase 1

1A:

```
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
heph@ravisvm:~/PES1UG24CS603-pes-vcs$
```

1B:

```
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
```

Phase 2

2A:

```
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B:

```
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ xxd .pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 | head -n 20
00000000: 626c 6f62 2036 0068 656c 6c6f 0a         blob 6.hello.
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ xxd .pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f | head -n 20
00000000: 626c 6f62 2036 0077 6f72 6c64 0a         blob 6.world.
```

Phase 3

3A:

```
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ echo "hello" > file1.txt
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ echo "world" > file2.txt
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ ./pes add file1.txt file2.txt
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  Makefile
  untracked:  tree.c
  untracked:  tree.h
  untracked:  .gitignore
  untracked:  pes.h
  untracked:  test_sequence.sh
  untracked:  test_tree
  untracked:  README.md
  untracked:  index.h
  untracked:  index.c
  untracked:  commit.c
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  commit.h
  untracked:  test_objects
  untracked:  test_objects.c
```

3B:

```
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ cat .pes/index
100664 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776451730 6 file1.txt
100664 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776451734 6 file2.txt
```

Phase 4:

4A:

```
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ ./pes log
commit e1beb1ff8a86ee6f9e2dcbd40d2abd2a86f2bf646f2a6be8e5bd2343dc752392
Author: Ravitejas <PES1UG24CS603>
Date:   1776452777

    Add farewell

commit 36078233ec29070581265a6057cc6f88bdeea8f69c65c108931b8aadd747f529
Author: Ravitejas <PES1UG24CS603>
Date:   1776452761

    Add worldell

commit 4124d72908502da9a9739cbee03750873661dafd1f778a23ac55126a8feed3ff
Author: Ravitejas <PES1UG24CS603>
Date:   1776452744

    Initial committial comQ
```

4B:

```
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/36/078233ec29070581265a6057cc6f88bdeea8f69c65c108931b8aadd747f529
.pes/objects/41/24d72908502da9a9739cbee03750873661dafd1f778a23ac55126a8feed3ff
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/96/c0d5c28dd3b719606c36723cd16aefbc73b3dc8b74de61070183357d6968b9
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/e1/beb1ff8a86ee6f9e2dcbd40d2abd2a86f2bf646f2a6be8e5bd2343dc752392
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/f1/0239b620372163c2dd06ac33189180c55edf1c56b15990a1cf44e00b1b0e59
.pes/refs/heads/main
```

4C:

```
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ cat .pes/refs/heads/main
e1beb1ff8a86ee6f9e2dcbd40d2abd2a86f2bf646f2a6be8e5bd2343dc752392
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Final:

```
heph@ravisvm:~/PES1UG24CS603-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: feb3fe260b9d... Initial commit

Log after first commit:
commit feb3fe260b9d2d92a849acda539365275e2ce55d58246d1849db0cd338aa53cf
Author: Ravitejas <PES1UG24CS603>
Date:   1776452901

    Initial commit


--- Second Commit ---
Committed: 9ac584d92933... Update file.txt

--- Third Commit ---
Committed: f523adbc1207... Add farewell
--- Full History ---
commit f523adbc1207cdbe6fcceedb0c604c6ad137f7597898141726470e1a7b994591
Author: Ravitejas <PES1UG24CS603>
Date:   1776452901

    Add farewell

commit 9ac584d929331c87487cde113eb5697005e4f8508600f7d63805fad79509a635
Author: Ravitejas <PES1UG24CS603>
Date:   1776452901

    Update file.txt

commit feb3fe260b9d2d92a849acda539365275e2ce55d58246d1849db0cd338aa53cf
Author: Ravitejas <PES1UG24CS603>
Date:   1776452901

    Initial committial comQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
f523adbc1207cdbe6fcceedb0c604c6ad137f7597898141726470e1a7b994591

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/9a/c584d929331c87487cde113eb5697005e4f8508600f7d63805fad79509a635
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/f5/23adbc1207cdbe6fcceedb0c604c6ad137f7597898141726470e1a7b994591
.pes/objects/fe/b3fe260b9d2d92a849acda539365275e2ce55d58246d1849db0cd338aa53cf

=== All integration tests completed ===
```

Phase 5 and 6 - Analysis only Questions:

Branching and Checkout

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

Ans: In our VCS, a branch is like a pointer: it's a file that has a commit hash in it. We need to change the .pes/HEAD file so that it points to the branch reference we want (for example, ref: refs/heads/main) in order to use pes checkout <branch>. After the pointer is updated, the working directory needs to be synced to match the state of that branch.

To do this, you need to go through the target branch's root tree object over and over again. The synchronization logic makes things complicated. We need to make files that are in the target branch but not in our current directory, delete files that are no longer there, and replace files that have changed. We must put in place a "safety gate" that stops the checkout if the user has uncommitted "dirty" changes, just like a naive checkout would.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

Ans: We check the metadata and content of a "dirty working directory" in two steps, using only the index and the object store:

Unstaged Changes: We check the current file metadata (mtime and size) in the working directory against the records in the Index. If they are different, the user has changed a tracked file but hasn't run pes add yet.

Staged Changes: We look at the hashes in the Index and compare them to the hashes in the tree of the current HEAD commit. If the hash in the Index is different, the user has made a change but hasn't run pes commit yet.

The directory is "dirty" if either check shows a difference. By not letting the user switch branches in this state, we keep them from losing data by accident.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

Ans: When the .pes/HEAD file has a raw commit hash instead of a link to a branch file, it means that the HEAD is "detached." In this state, if you make a commit, the VCS still makes the commit object and updates HEAD to point to it. However, no branch pointer (like main) moves forward to track it. These commits don't really have names. When the user checks out a different branch, the commits made in the detached state become orphaned. They stay in the object store but are no longer in the standard log. To get them back, a user would have to find the commit hash in their terminal history and then make a new branch pointer at that hash by hand.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

Ans: As time goes on, the object store fills up with "dangling" objects from deleted branches or commits that were never finished. We use a Mark and Sweep algorithm to clean this up:

Mark Phase: We start with all the "reachable roots," which include the HEAD, all the branches in refs/heads/, and any tags. We follow each link (Commits → Trees → Blobs) over and over, adding each hash we find to a Hash Set data structure.

Sweep Phase: We go through each file in .pes/objects one by one. We delete any file whose hash is not in our "Reachable" set because it is garbage.

We can't just look at the commits in a repository with 100,000 commits and 50 branches. We have to look at every unique tree and blob version as well. We would probably have to go through hundreds of thousands of objects to make sure we don't delete shared files that are still being used in other parts of the history.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

Ans: Running garbage collection at the same time as a commit is risky because of possible timing race conditions. For instance, a user might run pes add, which adds a new blob to the object store. The GC will see the blob as "unreachable" and delete it if it runs at that exact moment, before the user runs pes commit to link it to a tree. When the user finally tries to commit, the commit object will point to a hash that is no longer on disk, which will damage the repository.

In the real world, Git avoids this by giving people a grace period, which is usually 14 days. It will only remove objects that are older than a certain "last modified" timestamp and that can't be reached. This makes sure that new, unlinked objects are safe from deletion while they are being staged.